COSMIC SHORE · ENGINEERING

How we built unbreakable multiplayer

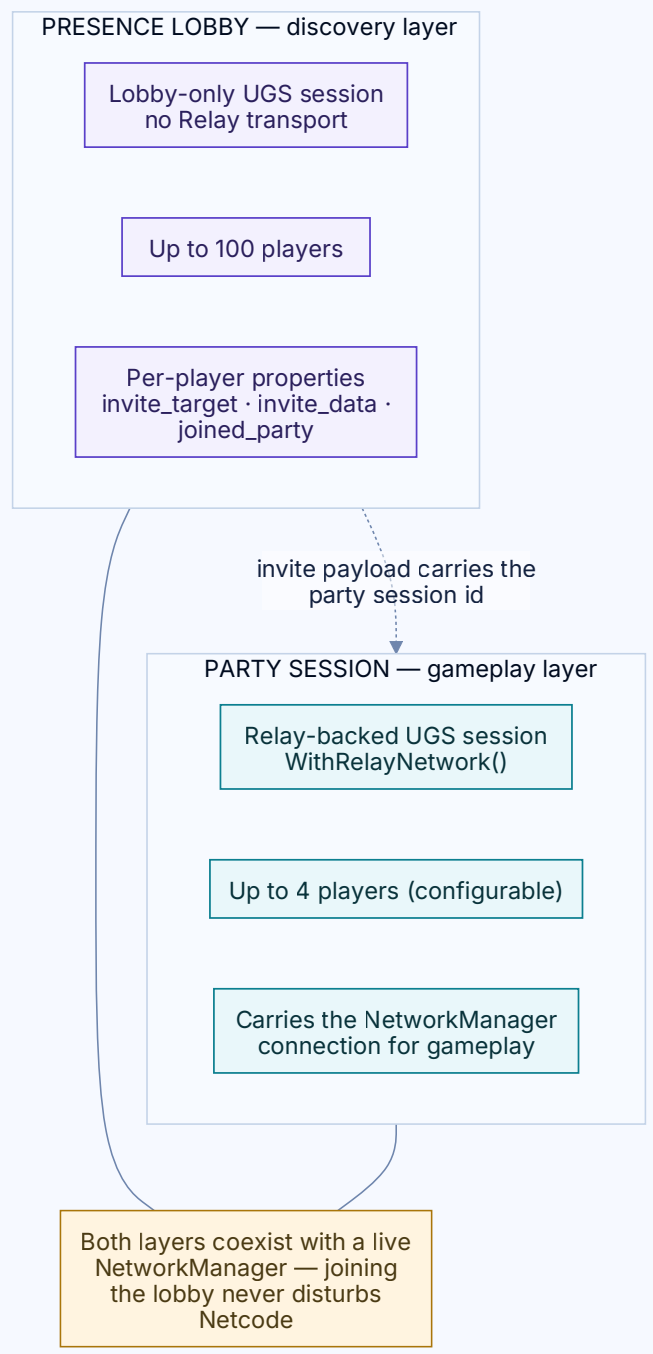
A plain-language tour of Cosmic Shore's online stack — Unity Netcode + Unity Gaming Services: the two-level session model, the decisions it hangs on, and the five bugs that taught us the most.

Swipe →

Multiplayer is really three hard questions

Where does shared state live? Which thread is your cloud callback on when it returns? And what do you do when two clients disagree about who's in the party? Answer them well and it feels seamless — answer them badly and it's endlessly flaky.

Split discovery from gameplay



A lobby-only **Presence Lobby** finds players cheaply; a Relay-backed **Party Session** runs the actual game. The invite is the bridge between them.

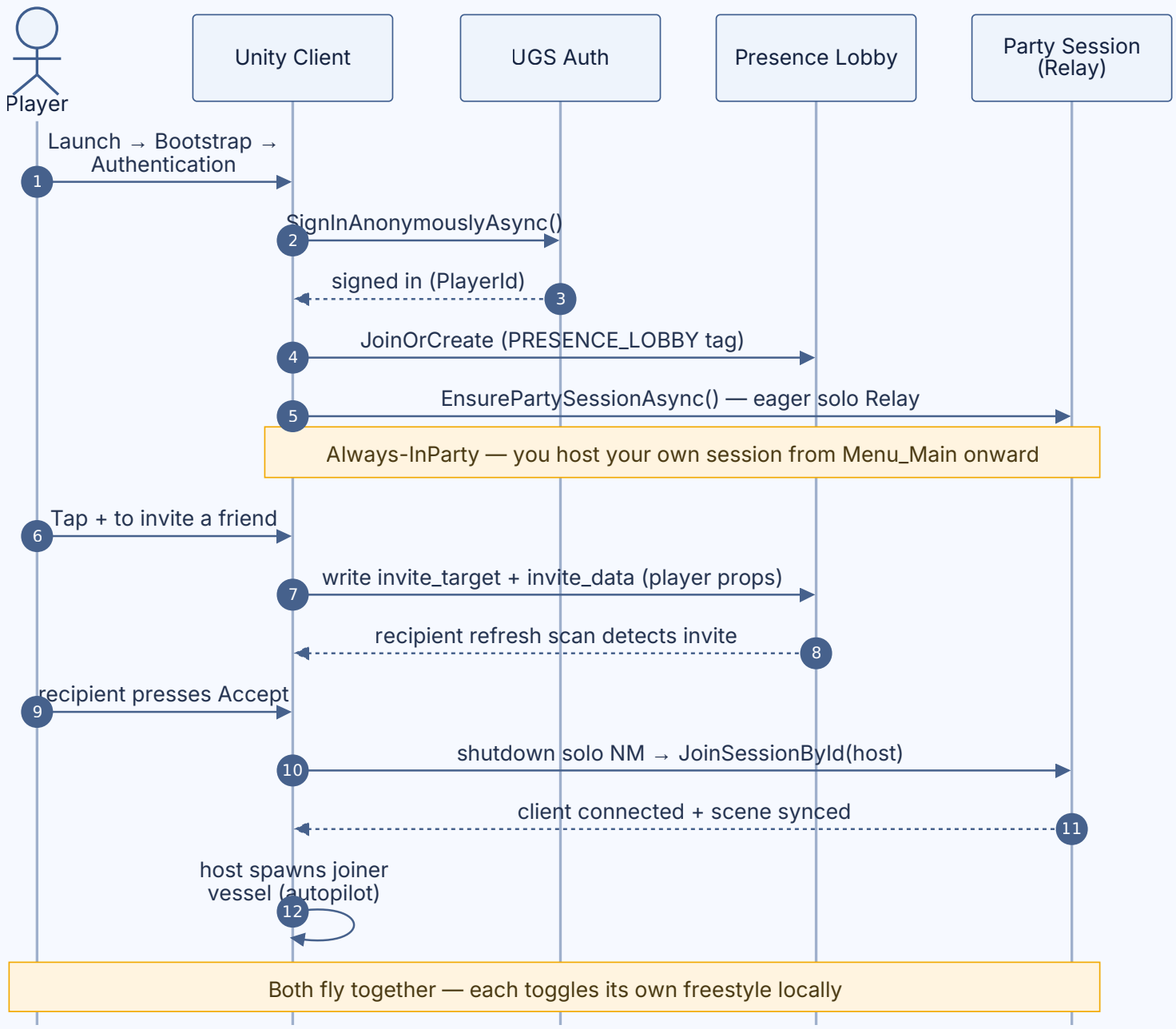
Cheap to discover, small to play

- **Presence Lobby** — up to 100 players, no Relay, and it never disturbs an active NetworkManager.
- **Invites need no host** — they ride per-player lobby properties, so anyone can invite anyone.
- **Party Session** — Relay-backed, capped, and exists only for the people actually flying together.

Everyone hosts a party from the start

EAGER “Always-InParty”: you create your Relay session the moment you reach the menu. So an invite becomes a simple **join** — not a fragile create-then-hand-off. An entire class of race-condition bugs simply disappears.

An invite is just a join



Because a real session already exists, accepting is “leave mine, join yours” — no session being built while another player waits to connect.

Three more things that keep it honest

- **Single-writer SOAP** — one owner per piece of shared state; everyone else reads.
- **A validated 7-state machine** — not a drift-prone scatter of boolean flags.
- **.AsMainThread() at every cloud await**, and server-authoritative spawning.

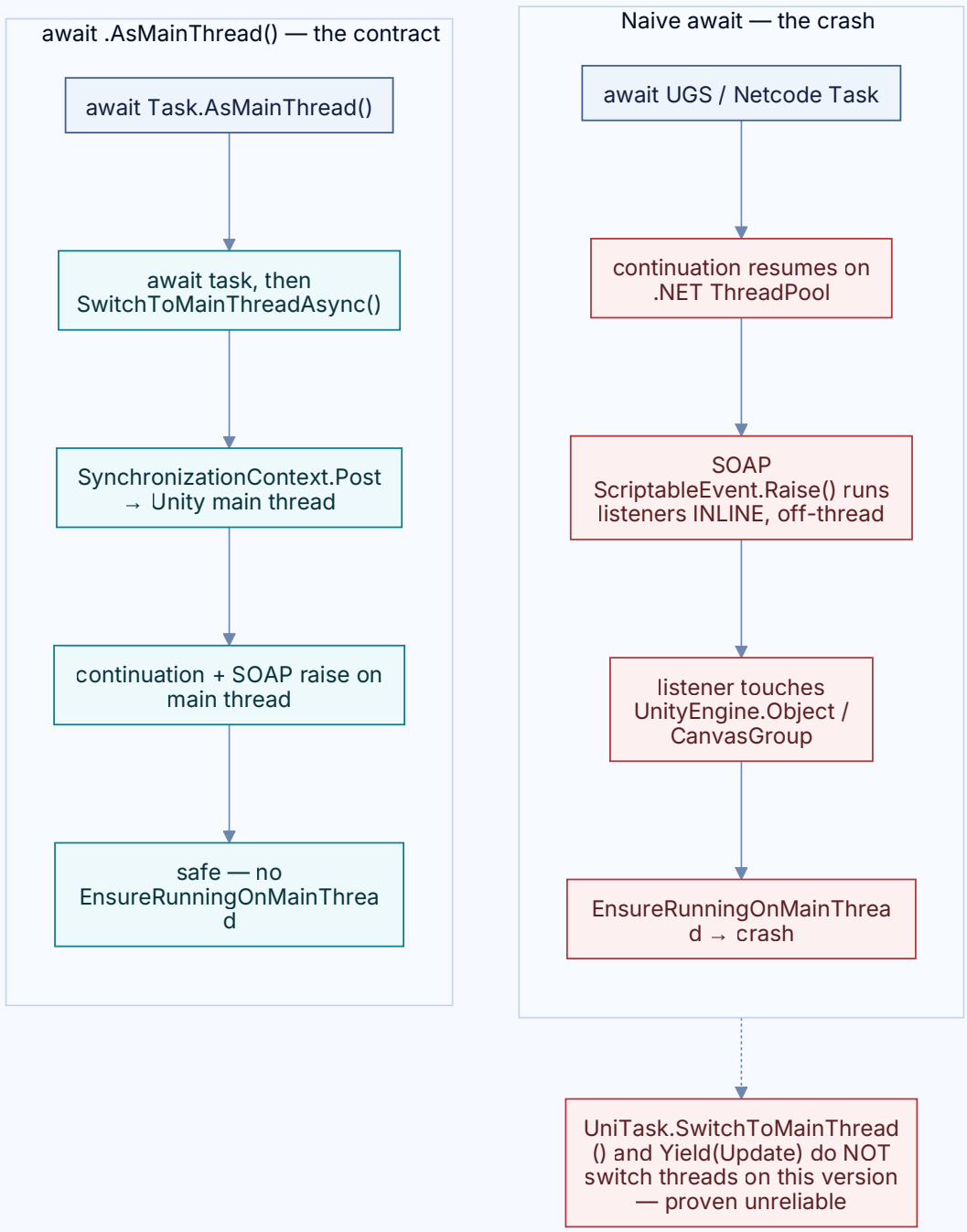
Your await came back on the wrong thread

Fixed

UGS callbacks resume on the **thread pool**. Touch any Unity object there — even `obj == null` — and it crashes. Worse: the obvious fix, `UniTask.SwitchToMainThread()`, was a **no-op** on our version.

```
// back on Unity's main thread:
var s = await Multiplayer.Instance
    .CreateSessionAsync(opts)
    .AsMainThread();
```

.AsMainThread() everywhere



A boundary helper built on Unity's own SynchronizationContext — plus a canary that screams if anyone forgets it.

A singleton pinned to null — forever

Fixed

Caching `MultiplayerService.Instance` in a constructor that runs **before** UGS initialises pins **null** permanently. Resolve it lazily instead.

```
//     ctor runs before UGS init → null forever
//     resolve at use time:
private IMultiplayerService _svc ⇒
    MultiplayerService.Instance;
```

When two truths disagree

Fixed

After a player left, the host kept **flickering them back** every few seconds — a presence-lobby “hint” was overriding the authoritative session. Fix: pick one source of truth (the session) and make the hint defer to it.

Two ships and dead controls

Fixed

Leaving a party spawned a new vessel **before** the menu finished reloading — so the reload spawned a second one, and the first was orphaned with no controls. Object and scene lifecycles are different clocks; order them explicitly or you get orphans.

8 “unbreakable” criteria — every commit

- No fatal failure · no stuck UI · no silent state divergence
- Every transition reversible · idempotent retries (double-tap is safe)
- 3- and 4-player concurrent-invite tests green in Multiplayer Play Mode

What we'd improve next

- **Host-loss resilience** — today the host is a player; if they drop, the party ends.
- **Prove 3–4-player parties** — close the second-joiner edge case.
- **Push over polling** — event-driven invites to cut latency + rate-limit churn.
- **Production telemetry** — measure party success and join latency in shipped builds.

5 reusable lessons

- Delete a fragile transition by making its precondition **always true**.
- Verify your async library's thread-switch — **don't trust it**.
- Never cache a singleton before its init can complete.
- One authoritative source; everything else **defers**.
- Order object vs. scene lifecycles explicitly, or get orphans.

COSMIC SHORE

The party game for pilots

Built by **Froglet Inc.** on Unity 6, Netcode for GameObjects & Unity Gaming Services.

Found this useful? Follow along for more engineering deep-dives.